California Polytechnic State University, Pomona
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Open Source SCADA and DCS System on Raspberry Pi Model 4 and Arduino Mega 2560 R3

Senior Project Semester 2
EGR 4830 Spring 26

Supervised by:
Professor Olson

Prepared by:
Brandon Esebag, Karina Francis, Marlene Pimienta Herrera,
Carlos Chavez, Armando Rodriguez, Steven Soto

Date: Thursday, 5/14/2026

CONTENTS

Open Source SCADA and DCS System on Raspberry Pi Model 4 and Arduino Mega 2560 R3

Brandon Esebag, Karina Francis, Marlene Pimienta Herrera, Carlos Chavez, Armando Rodriguez, Steven Soto
biesebag@ieee.org, karina.marie.francis@gmail.com, xx, Carlosc3980@gmail.com, xx, stevensotowk220@gmail.com
Department of Electrical and Computer Engineering
California State Polytechnic University, Pomona
Pomona, CA, USA

***Index Terms*—Raspberry Pi, Arduino, SCADA, DCS, Wi-Fi, Consumer-Grade, Automation**

***Abstract*—This project presents the design and implementation of a low-cost Supervisory Control and Data Acquisition (SCADA) and Distributed Control System (DCS) using consumer-grade hardware. The system recreates core functions of commercial industrial automation platforms by integrating Arduino Mega 2560 R3-based field devices with a Raspberry Pi Version 4 host responsible for system supervision, data processing, and user interface delivery. The architecture enables real-time monitoring and control while remaining significantly more affordable than traditional PLC-based solutions. The system's performance demonstrates that reliable small-scale automation is achievable using widely available components, making the design suitable for educational, prototyping, and non-industrial applications. Ethical and professional considerations—including environmental impact, user safety, and cybersecurity limitations—are analyzed in accordance with the IEEE Code of Ethics. The results show that while consumer-grade hardware imposes clear limitations, the proposed system provides an accessible and economical platform for learning and experimentation with SCADA and DCS technologies.**

I. Introduction

Supervisory Control and Data Acquisition (SCADA) systems are ubiquitous in industrial automation and utility management, and in recent years similar technologies are finding uses in consumer applications such as "smart homes". Direct Control Systems (DCS) typically use a computer to directly control a plant (such as an HVAC system), these systems are typically smaller in scope than the SCADA system(s) they interact with.

A. Project Overview

As part of a Senior Design Project (SDP), this paper outlines the production of a simple and scalable Wi-Fi-accessible SCADA system that can supervise numerous simple Direct Control Systems. The SCADA implementation is deployed on a Raspberry Pi Model 4, while each DCS is implemented using an Arduino microcontroller. Communication between the SCADA system and each DCS is established using USB serial connections (USB A to USB B). The SCADA software can issue commands such as modifying setpoints or receiving real-time telemetry data from all connected DCS units, and will interface with an application which can be run on a separate device over Wi-Fi to allow wireless communication with the SCADA system.

B. Public Health, Safety, and Welfare Impact

The development and potential future manufacturing of this SCADA and DCS system would involve the same externalities that come with the manufacturing of its components.

1) PCB Manufacturing Energy Consumption: "Manufacturing PCBs consumes a notable amount of energy, particularly during high-temperature processes like reflow soldering. These ovens and other assembly equipment often rely on non-renewable electricity sources, further amplifying the pcb environmental impact. Emissions are also a concern. Processes such as soldering and cleaning can release volatile organic compounds (VOCs) and other airborne pollutants." [1]. Although manufacturers are taking steps to reduce these concerns [1], the environmental impact(s) of PCB production are not insignificant. "Unfortunately, the impact of PCB manufacturing itself is just a small percentage of the impact created by the entire electronics industry." [2]

2) PCB Manufacturing Waste Production: PCB production also results in a significant amount of waste production "including trimmed materials, defective boards, and excess components." [1] This is a significant issue in the PCB manufacturing industry because "without proper waste management, PCB manufacturing contributes to hazardous waste which may affect human health and safety." [2] This is in part due to "[t]he assembly process for PCBs [which] involves a range of chemicals — from fluxes to cleaning agents — that can pose environmental risks. If mishandled, these substances may contaminate water sources or soil. Some also pose health risks to workers when not properly managed" [1].

3) PCB End of Life Concerns: As previously mentioned, PCB manufacturing produces significant electronic and chemical waste. "More concerning, however, is what happens when PCBs reach the end of their lifecycle. Improper disposal contributes to the growing global issue of e-waste, with toxic materials leaching into soil and groundwater" [1]. This issue can be partially mitigated by recycling, which is growing in popularity due to the presence of "[v]aluable metals such as gold, silver, and copper can be recovered from old boards." [1], as well as growing local requirements [1].

4) Cables Manufacturing Concerns: The manufacturing of the needed cables, such as serial cables (USB A-B) and power cables, comes with many of the same impacts that result from PCB manufacturing [3].

5) Logistics Concerns: All of the industrial activities mentioned above, and others (such as those needed to manufacture the sensors being used) require large quantities of materials to be transported during each supply chain. This is an environmental concern due to the required power-usage, the resulting emission, and other environmental costs of industrial logistics.

C. Global Impact

The global impact of this project is expected to be relatively small, as industrial automation using SCADA and DCS systems is already widespread. Moreover, the Raspberry Pi Version 4 and Arduino are consumer grade electronics and are not designed for industrial applications. As such, this system is better suited for consumer electronics or home automation scenarios as opposed to true industrial deployment.

D. Cultural and Societal Impact

A scalable, open-source, and user-friendly SCADA and DCS system deployable on consumer-grade hardware could have a wide-sweeping and positive social impact. Intended users, who are expected to have some familiarity with SCADA and DCS systems, would be able to implement these systems efficiently and at low cost. Potential applications include data collection and control for smart homes, small offices, non-industrial locations, or small businesses.

The system capabilities for each plant (DCS system) are limited only by the limitations of the Arduino, which supports a substantial range of simple or DIY applications. Compared to commercial SCADA systems, such a setup is significantly less expensive than a traditional PLC based setup, increasing accessibility for scenarios with limited budgets.

E. Environmental Impact

The environmental impact of this project was addressed extensively in Section I-B, where it was stated that this project would have the same environmental impact as the production of all the standard components within it. The Raspberry Pi Model 4, the one or multiple Arduino boards, and all the cables required to configure this SCADA and DCS are environmentally costly to produce (as are any consumer electronics).

F. Economic Impact

This project is not expected to have a significant economic impact on existing SCADA or DCS providers. As established in Section I-D, this system cannot be deployed in industrial environments because the electronic components used are consumer grade. Consequently, the system is better suited for small-scale, educational, or low-cost applications, and is unlikely to cause any meaningful market disruption.

The hardware required to implement the SCADA and DCS systems developed in this project is inexpensive and widely available. Both Raspberry Pi Model 4 units and Arduino boards are produced at scale and can be obtained at low cost, supporting the accessibility of this design.

G. Ethical and Professional Responsibility Judgment

1) Environmental vs Economic Impact: The affordability of this SCADA and DCS system relies on low-cost PCBs and wiring, which may come from environmentally unsustainable manufacturing processes. IEEE Code of Ethics item 1 highlights the need to consider these impacts:

- This raises concern in accordance with the 1st IEEE Code of Ethics, "to hold paramount the safety, health, and welfare of the public, to strive to comply with ethical design and sustainable development practices, to protect the privacy of others, and to disclose promptly factors that might endanger the public or the environment" [4].

While inexpensive components may pose environmental concerns, the impact is comparable to that of common consumer-grade electronics using similar parts. Responsible sourcing and transparent documentation can help mitigate this concern.

2) Accessibility vs. Safety: Making this SCADA and DCS system inexpensive and accessible also increases the risk of inappropriate or unsafe use, particularly by individuals without adequate technical background or for industrial uses. This concern is covered by the same IEEE Code of Ethics item 1 [4]. Additionally, item 5 is relevant:

- The 5th IEEE Code of Ethics, "to seek, accept, and offer honest criticism of technical work, to acknowledge and correct errors, to be honest and realistic in stating claims or estimates based on available data, and to credit properly the contributions of others;" [4]

To comply with these requirements, the project's GitHub repository and application client must clearly state that the system is intended for non-industrial and hobbyist use only, and that risks such as fire or electrical hazards are the responsibility of the user to manage. Transparency about limitations is essential to meeting the obligation to be "honest and realistic in stating claims".

3) Open-source Transparency vs. Security Risks: The open-source nature of the project's software paired with the affordability of its required components enables rapid adoption of this SCADA and DCS system, but also exposes users to potential cybersecurity threats, including unauthorized access or data breaches. IEEE Code of Ethics items 1 and 5 apply here [4]. Item 6 is also relevant:

- The 6th IEEE Code of Ethics, "to maintain and improve our technical competence and to undertake technological tasks for others only if qualified by training or experience, or after full disclosure of pertinent limitations" [4]

The contributors do not possess extensive cybersecurity expertise, and therefore cannot guarantee secure operation of the SCADA and DCS system, especially when exposed to Wi-Fi. Users integrating the project into their own environments must assume responsibility for securing their deployments. Full disclosure of these limitations is required to meet ethical obligations.

II. Background

A. DCS - Digital Control System

Before the wide scale availability of digital computer hardware, control systems were implemented in hardware and utilized the system's physical response to their environment as a means of basic control [5, p 4-6]. The utility of using digital computing to provide system control is apparent, digital computers are flexible, typically fast (relative to actuation) and in recent times cheap relative to a physical control system. "The use of physical computers in the loop yields the following advantages over analog systems: (1) reduced cost, (2) flexibility in response to design changes, and (3) noise immunity" [5, p. 579]. The size of computer hardware has substantially decreased in recent decades, and "[w]ith the development of the minicomputer in the mid-1960's and the microcomputer in the mid-1970's, physical systems no longer need to be controlled by expensive mainframe computers" [5, p. 578].

1) PLC - Programmable Logic Controller: In industrial control, DCSs are often implemented using PLCs, which "[are] digital, electronic device[s] designed to control machines and processes by performing event-driven and time-driven sequential operations. [They] can be programmed without special programming skills, and [they] can be maintained by factory maintenance technicians" [6, p. 432]. Many PLCs are configured to communicate with a distributed control system [7, p. 395]. "Programmable logic controllers (PLCs) can be considered to have three parts: the input/output section, the processor, and the programming device, or terminal" [8, p. 76]. As discussed in Section I PLCs are typically more expensive than consumer grade controllers (such at the Arduino AVR board family).

2) Arduino AVR - ATmega RISC Controller: Arduino is a widely used low-cost, consumer grade microcontroller family, notably containing the Arduino Uno Rev3 [9] and the Arduino Mega 2560 Rev3 [10]. These two controllers have many features that can be used to configure logical control loops for most simple applications:

1) Digital I/O Pins
2) Analog Input Pins (ADC) (can be used as Digital I/O)
3) Digital PWM Output Pins
4) 8-bit and 16-bit Timers
5) Timer Overflow and Comparison Interrupts
6) Digital Pin Driven Interrupts

These boards do have severe hardware limitations that prevent the use of complex programs as they feature low frequency microcontrollers (ATmega series) and very limited memory (model dependent). These boards can be programmed using the Arduino IDE or the Arduino IDE CLI tool (which can be called from another program) [11].

3) Data Collection: Typically Digital Control Systems both need to collect data from their environment and configure pin outputs to enact actuation of the equipment they control. "To get samples of a physical signal such as a position or velocity into digital form, we typically have a sensor that produces a voltage proportional to the physical variable and an analog-to-digital converter" [12, p. 102]. In the case of mechanical boolean inputs such as buttons and switches, or even electrically or mechanically thresholded analog inputs, digital I/O pins can be used for boolean data collection. Data collection using a digital controller will be subject to several key limitations:

1. **Sensor Calibration Errors:** Firstly the data being collected by the DCS can itself be erroneous, electronic or mechanical sensors, especially budget friendly or consumer quality sensors, can themselves be inaccurate. Depending on sensor position relative to the controller, and the quality and thickness of the wires connecting it to the controller, interference may also disrupt the signal.
2. **Controller Calibration Errors:** The controller itself may also be miscalibrated, such that even accurate sensor readings will be incorrectly recorded. An example of this error type would be zero offset error [7, p. 334] which occurs when an analog input is arbitrarily offset during conversion.
3. **Sample and Hold:** Sample and hold involves polling an input at uniformly separated times t seconds (or ms) a part, these samples are the assumed value of that input for the remainder of the next duration t [12, p. 103]. This results in sampled data which unlike truly analog data, can be stored using finite storage space.
4. **Sampling Rate:** The sampling rate of a given sensor must be at least twice the value's expected maximum changing frequency to avoid aliasing [12, p. 103-109]. This provides a lower limit for practical sampling rate.
5. **Quantization:** Like sampling, quantization is a method of data reduction [12, p. 324] [7, p. 332]. When a signal is quantized, one chooses n bits to represent each sample which allows 2^n discrete values to be distinguished. The signal is then *rounded* to one of these values. Whether this rounding is performed via closest match, round-up or round-down depends on the hardware specifics of the Analog-to-Digital Converter being used.

From this it should be observed that any data recorded by the controller itself is subject to scrutiny meaning any data logging performed by a supervisory system will have data limitations and will likely also have data errors.

B. SCADA - Supervisory Control and Data Acquisition

Supervisory Control and Data Acquisition refers to a system that can both implement control over some process(es) and collect data from said process(es). "The digital computer can perform two functions: (1) supervisory[,] external to the feedback loop; and (2) control[,] internal to the feedback loop" [5, p. 578]. There are two major components to a SCADA system, as outlined below. For the purposes of this project, SCADA systems will be considered in the context of controlling one or more DCSs (using an Arduino AVR board) and collecting data from those DCSs. Since this system should be able to run on standard consumer hardware, it will consist of only software.

1) Supervisory Control: To implement software control over an Arduino AVR powered DCS, communication with the DCS must be established. This can be achieved over USB serial as Arduino AVR boards have USB serial capabilities [9] [10]. There are two possible implementations for this control, they are named in accordance with how they function:

1. Flash Control: An instruction from the SCADA software is converted into Arduino IDE compatible C++ code and uploaded to the Arduino. This instruction is persistent and will survive unpowering of the DCS.
2. Serial Control: An instruction from the SCADA software is sent over USB serial as a string which is interpreted by the Arduino. This instruction is volatile and will be reset to the default value if the controller loses power.

Though it may be possible to enact supervisory control over an Arduino AVR board by other means than USB serial, such considerations are outside the scope of this project.

2) Data Acquisition: Data acquisition from the perspective of the supervisory system is subject to key limitations:

1. **Serial Bandwidth:** A serial connection to the DCS can only transmit a limited amount of data in each direction over each second. This can be modulated by baud rate and polling delay, however the amount of data that can be transmitted is limited by the USB serial controller on the DCS as well as the USB serial controller on the SCADA system's host machine.
2. **Polling Rate:** The supervisory system must poll each DCS for data at some interval T, each T all available packets are obtained and stored. The rate this can be achieved will be limited by the speed of the database being used to store data (if data storage is implemented) and the duration of time needed for all variables to be transmitted from the DCS to the supervisor.
3. **Packet Corruption:** If the supervisory system beings polling a connected DCS during transmission of a packet (containing some data, maybe a variable and its current value), only part of this packet is captured leading to data corruption.
4. **Packet Loss:** If the serial connection is overburdened (likely from excessive baud rate or insufficient DCS-side transmission delay) packets may be lost.

The result of these limitations is that the data stored by the supervisory system will likely have missing or corrupted points in each series, and may miss sudden changes in DCS sensor states (thus not recording those events).

III. Related Works

A. Current Open Source SCADA Solutions

1) RapidSCADA: "RapidSCADA is an open source industrial automation platform" [13] that provides the tools needed for creation, monitoring and control for potentially large systems. "Rapid SCADA is a perfect choice for creating large distributed industrial automation systems. Rapid SCADA runs on servers, embedded computers and in the cloud" [13]. RapidSCADA can run industrial automation systems, internet-of-things projects, and direct control systems. RapidSCADA is built to run on large-scale server hardware, which makes it computationally inaccessible for low-budget or low complexity project.

2) OpenSCADA: OpenSCADA is another open-source framework for industrial process monitoring and control "and HMI (Human-Machine Interface) systems" [14]. It emphasizes modularity, offering separate modules for data acquisition, visualization, database logging, scripting, protocol translation, and device communication. OpenSCADA is often used as part of larger automation ecosystems and supports industrial-grade protocols and hardware interfaces. Its architecture is designed for long-term reliability, making it suitable for factories, utility systems, and distributed monitoring networks. Unlike our project OpenSCADA requires significant compute resources to run and significant technical knowledge to configure properly.

3) Proposed Solution: Both Open Source SCADA Solutions above emphasize industrial applicability, feature richness and significant scalability. The system in this project does not aim to compete directly with full-featured platforms like OpenSCADA and RapidSCADA. Instead our project prioritizes ease of deployment on low-cost hardware such as a Raspberry Pi and Arduino, enabling experimentation and small-scale automation tasks with as little knowledge as possible.

B. Current Open Source DCS Solutions

1) RapidSCADA: As described in Section III-A1, RapidSCADA [13] is capable of running DCS systems, however the hardware requirements for such a system are not accessible for low cost and low complexity applications.

2) TANGO Controls: "Tango is an Open Source solution for SCADA and DCS. Open Source means you get all the source code under an Open Source free licence (LGPL and GPL). Supervisory Control and Data Acquisition (SCADA) systems are typically industrial type systems using standard hardware. Distributed Control Systems (DCS) are more flexible control systems used in more complex environments" [15]. Tango requires an operating system to install, meaning that any DCS controller device must be sufficiently powerful.

3) Proposed Solution: The DCS system we propose is designed to run on the Arduino hardware. This Arduino model is an affordable, available, and easy to configure platform for low-compute projects. All other commonly cited open source DCS applications require an operating system for installation, our DCS implementation will run directly on the Arduino UNO/MEGA Rev3's ATmega328P microcontroller.

IV. Requirements and Specifications

Our SCADA and SDA systems were developed under many self-imposed design constrains, all of which were met during development. This system was publicly demonstrated at the 2026 Cal Poly Pomona Senior Project Symposium in Building 9, 401.

Hardware Constraints

1. The system should be able to maintain a water level within a user specified tolerance.
2. The system should be able to detect water level using an analog sensor.
3. The system should be able to actuate motors and valves to control the water level and water movement.

Manual Control Constraints

1. The system should have physical switches that allow hardware actuators to be enabled or disabled.
2. The system should have an emergency off switch to kill all actuator power without unpowering the SCADA system.
3. The system should have software override capabilities (software point holds) that allow the user to hold a variable (such as a sensor reading or output) at a certain specified level.

Internal Control Constraints

1. Each DCS should be able to control its actuators based on sensor input and user configured logical conditions.
2. All intermediate variables should be reported to the SCADA system for logging and display.

Communication Constraints

1. The SCADA system is connected to all active DCS systems via USB serial connection.
2. The SCADA system can be accessed over local WIFI for control and data retrieval.
3. The SCADA system can be accessed by SSH for headless control and CLI usage.

Software Constraints and Requirements

1. The SCADA software will feature both CLI and GUI user control that can be accessed locally or over the web (within local network).
2. The SCADA software will be self-deploying when ran, creating all folders and needed files at a user specified file location which can be moved.
3. The SCADA software will be multi-threaded such that user commands do not block server/supervisor operations.
4. The SCADA software will feature a **serial monitoring** thread to make serial connection management and detection non-blocking.
5. The SCADA software will feature a **FLASK back-end** thread to make user input and requests non-blocking.
6. The SCADA software will feature a **flashing** thread to make writing code to Arduino AVR boards non-blocking. This will use the Arduino IDE CLI.
7. The SCADA software will feature a feature complete code-block logic system with C++ code-generation. This should feed into the **flashing** thread via an upload pipeline.
8. The SCADA software should feature a functionally complete CLI that can access server commands.
9. The SCADA software should feature a functionally complete GUI that can utilize server commands to display information to the user.

V. Design

A. Chosen Components

1) DCS Controller: An Arduino MEGA 2560 Rev3 was chosen as the DCS for this project. Initially we intended on using an Arduino Uno Rev3 however the device did not contain enough memory for us to provide it with the generated code (by 4%).

2) SCADA Supervisor: A Raspberry Pi v4 is being used as the SCADA controller, however a significant portion of testing was done using a Framework 16 Latop.

3) Tubing: Quarter inch thick, clear plastic tubing is used for all actuator connections where liquid is transported.

4) Container/Project Platform: A three compartment plastic divider is being used as the platform for this project. The middle compartment was removed and is being used as space to place the valves and flowrate sensors. Holes were placed in the bottom of the top container (the container that is being level controlled) with fittings added for hose connections.

5) Pumps: HiLetgo DC 12V water pumps were used to move water up (into the controlled vessel) and down (out of the controlled vessel).

6) Valves: 12V normally closed solenoid valves were used to stop siphoning when pumping water down, and to implement a manually controlled valve that allows water to be emptied from the upper container.

7) Water-Level Sensor: A ruler-style analog water level sensor is used to determine the water level of the container. This is the primary input for determining actuation.

8) Temperature Sensor: It was suggested that temperature would be a good metric to track inside our system, however due to scope reduction it was not implemented.

9) Flowrate Sensor: It was suggested that flowrate detection across our pumps would be a good metric to track inside our system allowing PID style control with feedback, however due to scope reduction it was not implemented.

B. Electronics

An Arduino is not typically capable of delivering sufficient current to actuation devices such as pumps and solenoids. To alleviate this issue, a 12V DC supply is used to power the Arduino (barrel connection) and a set of relays. These relays are what is used to drive the pumps and valves, the Arduino simply outputs (digital) to these devices at 5V. The Raspberry Pi is separately powered. The wiring was connected using Wago connectors, and utilized wires of various gauges.

C. Manual Switches

Several manual switches are present that allow a single valve to be manually controlled, and system power to be manually controlled. This is implemented using simple rocker-style switches.

D. Network

To isolate our network (for SSH and GUI connection purposes), a TP-Link nano-router was used as a network extender for my Laptop's hotspot. This allows any device connected to that hotspot to wirelessly command the SCADA system.

TABLE I
SENSORS CHOSEN FOR THE CONTROL SYSTEM.

Sensor	Item Name	Role	Implementation Details
Water Level Sensor	**Robot Contact Water/Liquid Level Sensor**	Detects if water in the tank reaches a high or low point. Prevents overfilling or running dry.	Optical sensor output (HIGH/LOW). Connect to GPIO input. Can trigger pump ON/OFF automatically.
Flow Rate Sensor	**1/4" Water Flow Sensor (Hall-Effect)**	Measures water flow rate in L/min to ensure the pump is moving water and to calculate total volume.	Generates digital pulses proportional to flow. Connect to GPIO input; count pulses using interrupt.
Temperature Sensor	**NTC Thermistor 10 kΩ MF52-103**	Measures the water temperature. Can detect overheating or environmental changes.	Analog sensor. Use an ADC module (e.g. MCP3008) to read voltage, then convert to temperature in code.

TABLE II
ACTUATORS USED IN THE CONTROL SYSTEM.

Actuator	Item Name	Role	Implementation Details
Water Pump	**HiLetgo DC 12 V 240 L/h Micro Brushless Pump**	Pumps water into or out of the tank. Controlled automatically based on level sensor input.	Connect through the Pump Controller (relay/MOSFET) that switches the 12 V power.
Hydraulic Valve	**1/4" DC 12 V 2-Way Normally Closed Solenoid Valve (WIC Valve 2ACK Series)**	Opens/closes to allow or stop water flow.	Controlled through the Valve Controller. When the coil is energized, the valve opens; otherwise, it closes.

TABLE III
MODULES INTERFACING THE RASPBERRY PI WITH ACTUATORS.

Controller	Connected To	Role	Implementation
Pump Controller	Water Pump	Acts as a switch driver—receives a 3.3 V signal from the Pi to toggle 12 V power to the pump.	Can be a relay board or a MOSFET circuit. Isolates the Pi from the high-power pump.
Valve Controller	Hydraulic Valve	Same principle as the Pump Controller, but for opening/closing the valve.	When Pi output is HIGH $\rightarrow$ coil energized $\rightarrow$ valve opens. When LOW $\rightarrow$ valve closes.

TABLE IV
SCADA / HUMAN–MACHINE INTERFACE COMPONENTS.

Component	Role	Description
Laptop / Computer	CLI or GUI	Acts as the human–machine interface (HMI) for remote monitoring and manual control.
Wireless Communication	Local Area Wifi	Connects the Raspberry Pi to the laptop using Wi-Fi. Can send real-time data or receive control commands.
Software Options	Data Visualization	**Flask Web Server:** Allows CLI or GUI to send commands to SCADA. **Plotly:** Allows GUI to poll SQL database to generate plots. **SSH:** Remote access for starting and troubleshooting server.

TABLE V
COMMUNICATION LINK OPTIONS BETWEEN THE RASPBERRY PI AND ARDUINO.

Link	Pros	Cons	Method
USB Serial (UART over USB) *(Recommended)*	Easiest; powered and isolated via cable; stable.	Requires a USB cable.	Connect Arduino to Pi via USB; use `/dev/ttyACM0` or `/dev/ttyUSB0` at 115200 baud.
I^{2}C (Pi master, Arduino slave)	Simple 2-wire interface.	3.3$\leftrightarrow$5 V level shifting; address management.	Pi SDA/SCL $\leftrightarrow$ Arduino A4/A5 (Uno); add bi-directional level shifter.
SPI	Fast throughput.	Chip-select management; level shifting required.	Pi SPI0 $\leftrightarrow$ Arduino SPI pins; level shifter required.
RS-485 (Modbus RTU)	Long cable runs; noise-robust.	Requires transceivers and protocol stack.	MAX485 modules, twisted-pair cable, Modbus library.

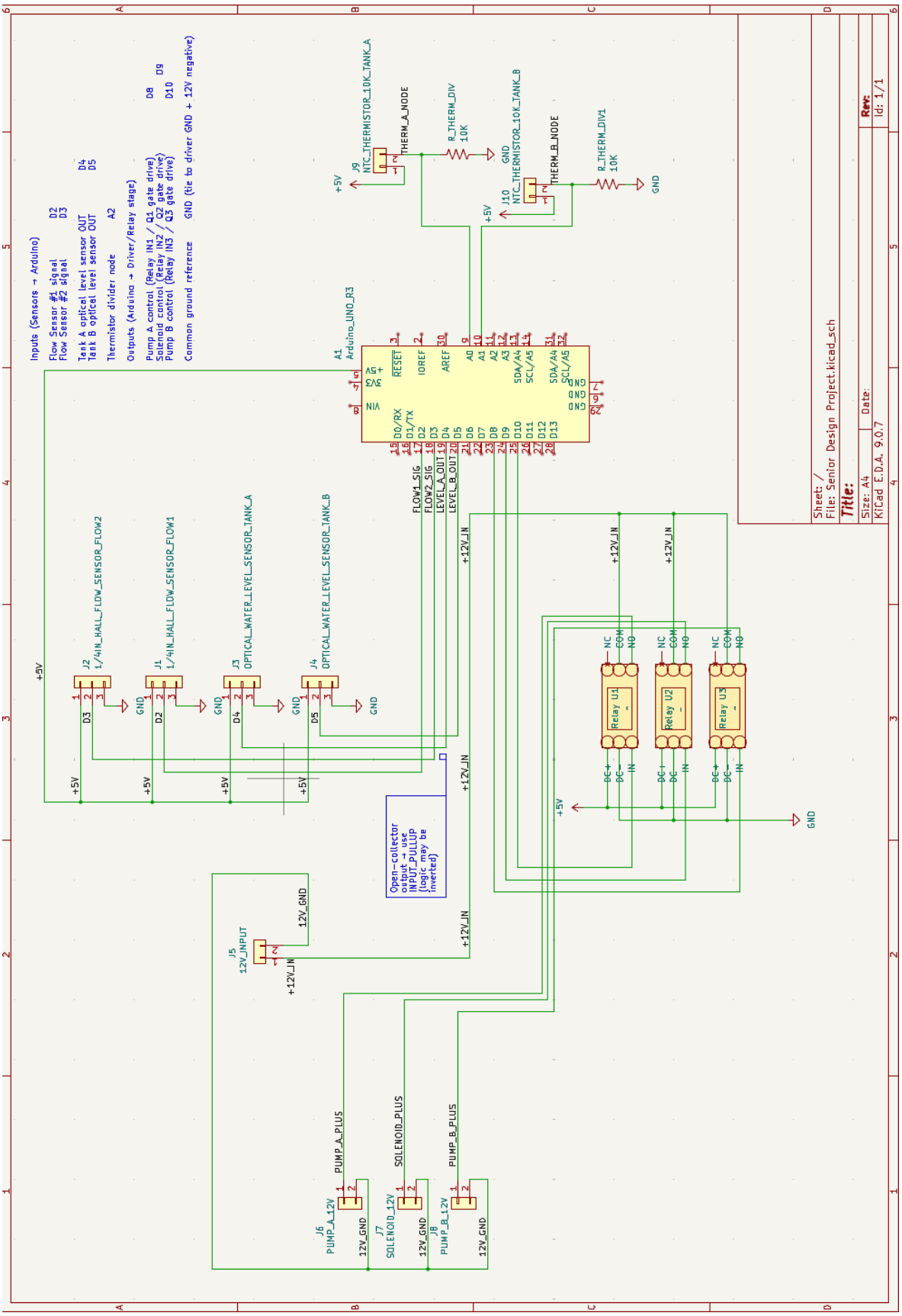

Fig. 1. Initial Hardware Schematic

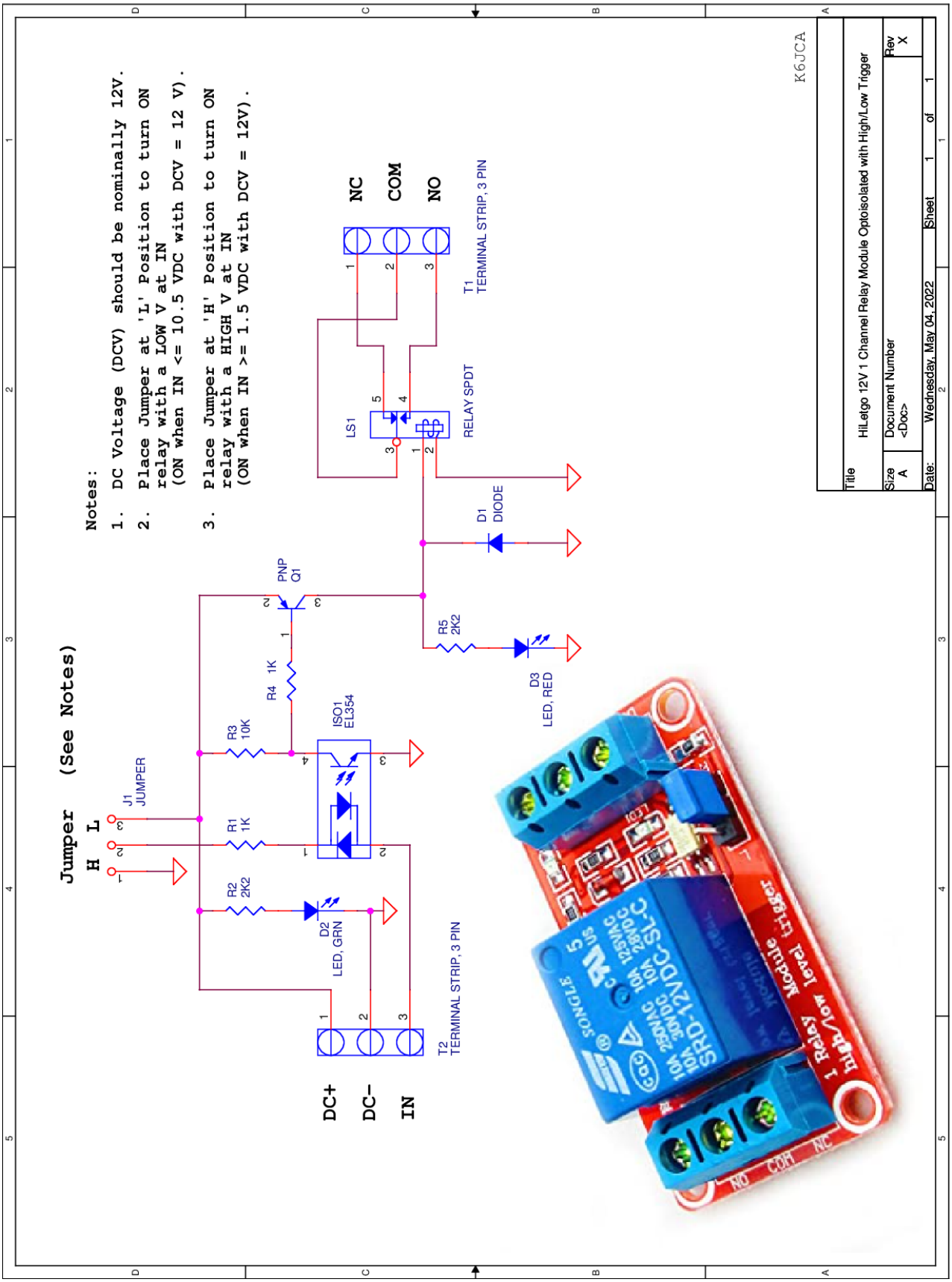

Fig. 2. Final Hardware Schematic

E. Software Design

The design of the SCADA software was done in piece-wise as new features were added, however the code-base utilizes multithreading to ensure all asynchronous operations are non-blocking to the main thread. The code-base is comprised of the following major sections.

1) Main Thread: The main thread (and entry point) is the contained in the executable SCADA.py. When this is initialized it will ensure a data path (outside of the repository path) is established for data and settings to be stored inside of on the host system. It will then initialize all other needed threads and facilitate pass-by-reference communication with and between all other threads.

2) Serial Thread: The serial (monitor) thread is the first thread to launch, it continuously monitors the host system's serial ports, updating the main thread when connection or disconnection events occur. This allows the main thread to run a set of functions for each event to determine whether the detection is an Arduino or if it should be ignored.

3) Flask Thread: The Flask thread launches a Flask backend-server that accepts POST commands (and filters inputs for invalid requests), this is the entry point to the software, both the CLI and the GUI simply access exposed commands in the Flask thread and it facilitates the rest of the SCADA-side logic. This thread also opens port 5000 for CLI and GUI connections.

4) Worker Thread(s): When a serial connection event is detected in the main thread, and this connection happens to be an Arduino, a worker thread is created for the Arduino' port. This thread both sends hold commands requested (via the Flask thread) and collects data at a user-specified polling rate and stores it in the SQL database (in the created data-path). This thread terminates itself when the Arduino is disconnected.

5) Controller Initialization: When a controller is connected for the first time, a JSON file containing all port configurations, user defined code blocks and settings is saved with default settings into the data path in a folder called "dcs_info". This and all settings modifications are defined in a python file called dcs_dict_utils which is by far the longest back-end file in this project. The functions in this section determine rules for pin and point interactions and settings, as well as the use of arrays, timers, ISRs and more. The combination(s) of settings achievable is detailed enough for code-generation, but there would be no implemented logic.

6) Code Blocks: Code blocks are semi-generically typed blocks that can be placed into any region in the controller's code, such as setup, loop, and inside of any ISR. The available ISRs are kept in a list (via many functions working in tandem within dcs_dict_utils) and update based on pin-driven interrupt settings and timer settings. Code blocks, such as the addition block, have some inputs (the two input numbers in this case), sometimes a number of outputs (in this case the sum), and a slot for a boolean variable that is used as a conditional statement. Some code blocks require an array as an input or output and cannot accept a point.

7) Points: Points as defined in the code-base are essentially software variables that can represent hardware (such as the points locked to pins or timer registers) or can be user defined (such as a user defined int, float, or bool). Points can be held over serial (via the worker threads) to a user-defined value, and points can be configured with a minimum and maximum. Internally to the Arduino, all points are a 5-value struct, but they are treated as a single variable using getter and setter commands (in generated code).

8) Code Validation: Before code can be generated, many simple rules are evaluated to determine whether the user-defined code-script makes sense. For example, a point being the output of two code-blocks causes a conflict, as such the validation will return a string listing that error. When an empty string is returned, the code-generation pipeline can proceed.

9) Code Generation: Using the configuration generated from the rules in dcs_dict_utils, and using the code-block logic defined in code_block_utils, the file ino_utils can be used to generate (mostly) feature complete Arduino AVR C++ code. All points and arrays get defined at the top and evaluated for whether they need to be constants or volatile based on the user-defined settings saved to file. Then a generic form of each code block (only those used at least once) is defined once in setup. Then in each section code-blocks were placed, a call to that block using the point getter function for each input (except for arrays) is used, and the outputs are assigned via the point setter function. Finally at the end of loop the Arduino is told to print all point values so that the worker thread can log them, and checks if the worker thread wrote a hold command which gets set to the specified point.

10) Uploading Code to an Arduino: Once an ino file is generated (which will share the same name as the controller in question), upon command the Flask server can push a queue request for a controller to be flashed by port name, this is passed to the main thread where the main thread passes it to both the worker in question and the flash thread. The worker must close its port before the flash thread can continue, but regains port control once flashing is complete (or fails).

11) Command Line Interface: During development and debugging the CLI was used to interact with the Flask server. Though the CLI is feature complete in terms of usability, it is far less pleasant than the GUI. The GUI was one of the last features added to the code, which meant the CLI received a lot of use. To use the CLI it can be run as a separate python file on the host machine, and when 'help' is entered it lists all valid commands.

12) Graphical User Interface: The GUI is a simple web-UI that is launched at localhost:5000 on the host machine, and can be accessed by typing "http://{ip}:5000" into the searchbar of any browser where {ip} is the local IP address of the host system, or "localhost" if launching from the host system. This GUI allows the code-block system to be used as a visual code editor, compiled and uploaded with the push of two buttons, and verified with live-updating and historical plotting for all connected controllers simultaneously.

VI. Integration and Test Results

To ensure all engineering efforts towards both the SCADA system and the DCS system would be compatible for simultaneous testing and use the two systems were continuously tested together during development allowing refinements on both fronts to be made. The DCS system was developed in hardware and was tested using temporary Arduino C++ code that would be used to calibrate the system and inform the development of C++ code-generation. The tests run on each subsystem as well as the resulting information will be separated by subsystem.

A. Hardware Development and Testing

Before testing any control circuitry, verification of the operation of the valves and pumps was performed using direct 12 V separated by a Single Pole Single Throw enable switch to turn the pumps on and off. A 12V 2A AC Adapter Power supply was used to power the actuators and was verified to have enough current to power 3 Actuators at once. From these tests, operation of the pump's intake and release from one tank to another was verified before being connected to any control circuitry.

1) Issues with Power Distribution: In the initial design, N-channel MOSFETs, pull-down resistors, and flyback diodes to switch inductive load were used to drive relays responsible for switching 12 V loads such as a solenoid valve and water pumps. However, this first iteration exhibited inconsistent behavior during testing. The relays failed to actuate reliably due to unstable gate drive and grounding issues, and in some cases only one 12 V load could be operated at a time. This indicated limitations in both the control circuitry and overall power distribution.

2) Debugging Power Supply Issues: To address these issues, a commercially available buck converter was introduced to regulate and step down the supply voltage for logic-level components, providing a more stable voltage source. While the converter was capable of supplying up to 2 A under max load, the increased complexity of the power distribution network introduced additional risk during testing. Although relay actuation became more consistent, the system still failed to reliably switch the 12 V loads, indicating that the underlying issue was not solely related to supply regulation.

The relays were intended to electrically isolate the 5 V logic domain from the 12 V actuator domain; however, the discrete implementation did not consistently achieve this objective. These limitations ultimately led to a redesign of the actuation stage using a commercially available AEDIKO DC relay module with proper integrated driver circuitry and protection components such as optocoupler isolation that provides driving ability and stable performance.

This modification simplified the circuit design while providing quality electrical isolation between the low-voltage logic control signals (5V) and high-power components (12V). Similarly the inclusion of internal protection elements such as flyback diodes and optocouplers, reduced voltage transients and improved switching stability while the onboard LEDs showcased the on-off states of the relay when providing power for the pumps. These alterations were able to enhance the SCADA system by improving robustness and reducing system complexity compared to integrating a custom relay PCB with limited knowledge of power electronics.

3) Worst Case Current Draw: In terms of powering all components in the project, a table below was created to determine the worst case for current draw of all components in the system. From this tabulation and consulting Arduino datasheet, it was determined that the total current of the system was too high for the Arduino to support, thus introducing the need for a dedicated 5V 1A AC adapter for the relays separate from the Arduino's 5V supply.

TABLE VI
Current draw of system components.

Part	I (mA)
Relay 1 ON	80
Relay 2 ON	80
Relay 3 ON	80
Flow Rate Sensor 1	10
Flow Rate Sensor 2	10
Water Level Sensor	5.2
Digital Output 5	5
Digital Output 6	5
Digital Output 7	5
Potentiometer	5
Total without Relays	45.2
Total with Relays	285.2
Maximum I of Arduino	150
Maximum I of Alt. Power Supply	1000

4) Testing of Water Level Sensor: Initial designs included both a binary water level sensor on each tank to indicate when a tank was full as well as an analog sensor for one main tank. This scope was then scaled down to just focus on a high quality Analog water level sensor as this sensor would act as the main feedback of the system. The water level sensor chosen was the eTape 12" Continuous Fluid Level Sensor PN-12110215TC-X. Our methodology while using the sensor included both multimeter testing of the resistance of the sensor in response to water level change and serial printing the analog value in the Arduino IDE serial terminal. Through repeated measurements we determined that the water level resistance ranged from 400 ohms to 2.2kOhms. For our purposes we primarily used resistances ranging from 1.2kΩ to 2.2kΩ.

This is consistent with the datasheet provided by the manufacturer as the tank we were using was 8" deep therefore only using about 75% of the total length of the sensor. This limited accuracy in the change of water level data but gave us more modulability if we were to change tank size. The output voltage sent to the analog input of our Arduino was obtained through a voltage divider of a known 560 Ω resistor and the water level sensor using the following equation, followed by

a simple example when 5V is input:

$$V_{analog} = Vin \cdot \frac{R_{sense}}{R_{sense} + 560\ \Omega} \tag{1}$$

$$V_{analog} = 5\ \mathbf{V} \cdot \frac{2.2\ k\Omega}{2.2\ k\Omega + 560\ \Omega} \approx 4.07\ \text{V} \tag{2}$$

This voltage was mapped onto Arduino analog values of about 5mV per analog unit. This calculation is based on the analog to digital step size of the Arduino. From testing of the water level sensor while connected to the Arduino IDE Serial Terminal, we obtained consistent analog values of 750 to 824 which when mapped onto the 5mV per analog unit is a range of 3.66 V to 4.03V using the following equation, followed by a simple example when 5V is input:

$$V_{analog} = \text{ADC}_{val} \cdot \frac{V_{in}}{1024} \tag{3}$$

$$V_{analog} = 824 \cdot \frac{5\ \text{V}}{1024} \approx 4.02\ \text{V} \tag{4}$$

To confirm this finding, these calculations can be run in the reverse direction, returning the original input voltage, shown below. From testing both physical resistance from multimeter and serially printed Analog Values we were able to verify and calculate the minimum and maximum ranges of our system's water level.

$$V_{in} = V_{analog} \cdot \frac{560\Omega + R_{sense}}{R_{sense}} \tag{5}$$

$$V_{in} = 4.02\ \text{V} \cdot \frac{560\Omega + 2.2\ k\Omega}{2.2\ k\Omega} \approx 5.04\ \text{V} \tag{6}$$

5) Testing of Flow Rate Sensor: While not used in the final implementation of the project, a flow rate sensor was also used which sent variable length pulses to the Arduino's digital input based on the rate of water flowing through the sensor. In order to verify results of the flow rate sensor and operational code we took timed trials of the system filling a 1.5 Liter bottle and divided the volume of the bottle by the time it took to fill the bottle in minutes to get the Liters per minute flow rate of our pumps. We verified that the pumps were operating far below the advertised 4 L/min and were consistently running at 0.78 L/min which was consistent with the numbers observed on the Arduino IDE Serial Terminal.

B. Trouble Shooting Test Control Code

Before integration of the initial Arduino C++ code with our water tank system, a simple Analog Potentiometer and LED indicators were used to represent the Water Level Sensor and Digital Outputs to Relays. This allowed for prototyping the code away from the main system. Modifications were added to the code including software delay functions, and increasing the tolerance for the analog value to reach the desired setpoint. Using minimum and maximum values obtained from testing of the water level sensor, a preliminary control code was constructed from this prototype that could interface with the water tank system.

Upon using the above mentioned C++ Arduino control code with our water tank system, noticeable flickering between states would occur and would be exacerbated by the ripple of the water due to the pumps. The sensitivity of the water level sensor was discovered to be too high especially when crossing the threshold to the setpoint's tolerance. To remedy this, a moving average function was added to the code which filtered out the flickers caused by the slight ripple of the water. Additionally an incrementing counter was added to allow the pump to remain on long enough for the water level to cross into the setpoint region before the system would shut it off. This reduced response time but allowed the system to become more stable instead of oscillating between shutting the pump on and off frequently which put strain on our hardware.

During testing the pump responsible for draining one tank was discovered to create a siphon that would continuously drain water from the tank even when turned off, to remedy this a Digitally controlled valve was added in line with the pump and was added as another Digital Output for the code, logically tied to the drain pump's digital output.

C. SCADA Software Development

The SCADA software, which was primarily developed by Brandon with experimental assistance from Karina, was tested at the addition of each feature (of which there are dozens, a full list of software features can be found in the help menu when the CLI is used) by connecting an Arduino board to the Framework 16 laptop and evaluating the new feature's performance and functionality. This was developed in much the same way most code is, through trial and error, and debugging, as such there is not one significant experiment that occurred like there was for hardware testing.

When C++ code generation was implemented it was initially non-functionally buggy, the testing that resulting in that being fixed incolved git-pulling the new code onto the Raspberry Pi v4, attempting code-generation and flashing, it fails. A USB is mounted on the Raspberry Pi (through SSH), the ino file generated by the SCADA software is copied onto the drive, then the drive is unmounted. Then the code is inspected on the laptop in the Arduino IDE. When an error is resolved the code-generator is updated, the code is pushed to GitHub and the cycle repeats. This activity took around 12 hours and was done in one sitting, but the code generator works.

VII. Standards and Constraints

Standard: International Society of Automation (ISA). ISA112: Supporting SCADA System Reliability. International Society of Automation, 2023.

Justification: We chose ISA-112 as it provides guidelines for the design, implementation, reliability, and maintenance of SCADA systems. Since this project is a low-cost SCADA system we decided to use the standard to guide the system's architecture and added features that resonate with ISA112. We added valves to our pumps to ensure there would be no risks of leaks, we also added an emergency dump valve to use if the system would ever fail, we had a manual way to dump the

water. Features such as these backups and valves, add a level of security to ensure if errors would occur, we had ways to deal with them.

Standard: IEEE 802.11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications. Institute of Electrical and Electronics Engineers, IEEE Standards Association.

Justification: This standard defines how wireless devices communicate over Wi-Fi networks. Originally since we wanted our project to be wireless, we would have followed this standard as it was made so different manufacturers can communicate reliably using the same networking rules. Although we ended up not following this standard due to us utilizing wired connections between the Arduino and laptop. If we had stuck to wireless communication, IEEE 802.11 defines the protocols to ensure wireless data transmission, which is important as a SCADA system heavily operates on moving data around.

Standard: IEEE Std 1615-2019, IEEE Recommended Practice for Network Communication in Electric Power Substations.

Justification: This standard details the use of TCP/IP Ethernet connections for communication in SCADA control systems. This standard recommends utilizing Ethernet switches and Serial-to-Ethernet converters in wireless networks and SCADA systems. This standard is suitable for our project as we were considering implementing Ethernet controlled DCS support and may consider it as a future work.

Standard: IEEE Std 3001.11–2017, IEEE Recommended Practice for Application of Controllers and Automation to Industrial and Commercial Power Systems.

Justification: This standard details selecting controllers and the application of controllers for industrial systems. For outputs where the pilot controllers supply less voltage than the load needs the standard recommends using relays for the outputs. I anticipate our project following this standard because our project will utilize 5 V for the outputs but most motors and valves require 12 V or 24 V. Therefore we will follow this standard and use a relay to drive the actuator.

VIII. Project Planning and Task Definition

During the project planning phase (Fall Semester, 2025) many of the project members had conflicting schedules and could not regularly meet. A significant portion of the work done during this phase was research and conceptual design, but hardware design, testing, and software design would not commence until Spring 2026. Due to this significant delay the project was scheduled and executed under severe time constraints.

Once regular contact was established between members, meetings were held each Monday from 11:00-2:00, 4:00-5:00 PM, and often on Wednesday at the same time from 11:00-2:00. Brandon chose to focus on the software that would be implemented for the SCADA system, testing it on both the Raspberry Pi and a laptop using several Arduino boards. This was done rather sporadically as bugs were regularly encountered and resolved. This effort was eventually unified with the rest of the project when C++ code generation was implemented. Because this task was relatively insular he did not interact with the team as regularly as the other members.

Karina, Steven and Carlos focused on leading the hardware effort, testing and adjusting circuitry and sensor readings using the test code Karina developed in parallel to the SCADA code (pre code-gen). The test code developed during these experiments informed how the code generator Brandon developed would have to work. Karina managed a schedule using Gantt charts, one of which is posted below. Marlene helped secure the fittings onto the container resulting in water-tight connections, as well as working on the spigot (for manual emptying) and the valves.

	Wednesday 3/18/2026	Sunday 3/22/2026	Monday 3/23/2026
Control Circuit	Start Building, issues with 12v supply		
Pump Control Code	Test it, works		
Water level sensor code (A)		Got min and max values, mounted	
Water level sensor code (D)			
LED indicators			
Power Bus	Switch test		
Flow Rate Sensor Code			
Flow Rate sensor Installation	Done		
Pump Hardware Test	Done		
Valve Hardware Test	Done		
Spigot	Research	Drilled hole, leaky	
Tank Water management	Research		
Water proof sealing	Ordered Silicon	Got it waiting	
Water level sensor test (D)			
Documentation			

Fig. 3. Gantt Chart used for Event Scheduling

Steven taught Carlos how to solder, as well as building the initial schematic that governed the arrangement of MOSFTs and other circuit components, eventually soldering them onto prototype board. It was eventually deemed that commercially available power circuits would be used for convenience. Marlene put a substantial effort into graphic design, producing the Symposium Poster used by our group.

IX. Conclusion

A. Project Results

This project successfully demonstrated the design and implementation of a low-cost, open-source SCADA and DCS system using a Raspberry Pi and an Arduino. By integrating these consumer-grade components, the system effectively recreated the core functions of commercial industrial automation platforms. The final architecture proved to be a highly accessible alternative to traditional PLC-based setups. Successful management of hardware and software constraints led to fast development and integration. The Raspberry Pi SCADA supervisor provided user friendly CLI and GUI interfaces over local Wi-Fi, along with dynamic code implementation capabilities. Operation was reliable and the Arduino based DCS successfully maintained a user-specified water level within a tank system by interpreting analog water level sensor data and actuating 12V DC pumps and solenoid valves.

Throughout development, several critical engineering challenges were navigated and resolved. Initial power distribution

and gate drive issues that hindered reliable relay switching were solved by replacing discrete MOSFET circuits with a commercial relay module featuring optocoupler isolation. Furthermore, control code instability caused by water ripple was effectively controlled by implementing a moving average function and an incrementing counter to stabilize sensor feedback. By incorporating safety redundancies like manual overrides and emergency dump valves, the design adhered to ISA-112 and IEEE recommended practices for system reliability. Ultimately, this project proves that while consumer-grade hardware falls short of the complexity needed in industrial use, it successfully provides a highly functional platform for learning and experimenting with SCADA and DCS technologies.

B. Future Work

There are several avenues in which this project can be improved and expanded upon in the future, besides software bug fixing.

1) Ethernet Connected DCS: A way that the communication protocol of the code-base could be expanded would be to add the capability for Ethernet-driven DCS communication as opposed to USB serial. This would allow the Arduino Ethernet Shield to be utilized.

2) Bluetooth Connected DCS: Another way that the communication protocol of the code-base could be expanded would be to add the capability for Bluetooth-driven DCS communication. This would similarly allow the Arduino Bluetooth module to be utilized.

3) Wifi Connected DCS: Another similar way that the communication protocol of the code-base could be expanded would be to add the capability for Wifi-driven DCS communication. This would represent all three layers of control being separated on a local network, and would require significant code refactoring and hardware addition.

4) Raspberry Pie DCS: The Raspberry Pi computers are much more powerful and much faster than the Arduino AVR boards, these could be used as DCS controllers instead of SCADA controllers in systems that need faster response or contain more information (more complex equipment).

5) Addition of more Code Blocks: The selection of code blocks is limited due to this software being experimental, expanding the default library of code blocks would be useful.

6) Code Block Editor: An editor in the CLI and GUI that would allow a user to make or edit code-blocks would also alleviate the issues of code-block sparsity in a more custom way.

Acknowledgment

The authors would like to thank Professor Olson for advising this Senior Design Project. All project participants contributed to and learned a lot from this project, and it would not have been possible without Professor Olson.

Project GitHub Repository:
https://github.com/rancorjoy/SDP_SCADA

References

[1] M. Media. (2025) Understanding the PCB environmental impact: What you need to know. Accessed: Dec. 4, 2025. [Online]. Available: https://novaenginc.com/pcb-environmental-impact/

[2] (2025) PCBS and the environment. Accessed: Dec. 4, 2025. [Online]. Available: https://www.pcbnet.com/blog/pcbs-and-the-environment/

[3] Q.-C. Wang, T. Lu, H.-S. Chen, L. Wang, J. Jia, and W.-Q. Chen, "Tracing environmental footprint of copper wire rod manufacturing in China," *Resources, Conservation and Recycling*, vol. 204, p. 107503, May 2024.

[4] IEEE. (2025) IEEE code of ethics. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8.html

[5] N. S. Nise, *Control Systems Engineering*, 8th ed. Hoboken, NJ: Wiley, 2019.

[6] R. N. Bateson, *Introduction to Control System Technology*, 6th ed. Columbus, OH: Prentice Hall, 1996.

[7] J. M. Jacob, *Industrial Control Electronics: Applications and Design*. Columbus, OH: Prentice Hall, 1988.

[8] T. J. Maloney, *Modern Industrial Electronics*, 4th ed. Columbus, OH: Prentice Hall, 2001.

[9] Arduino, *Arduino Uno Rev3*, Arduino SA, 2026, accessed: 2026-05-14. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3/

[10] ——, *Arduino Mega 2560 Rev3*, Arduino SA, 2026, accessed: 2026-05-14. [Online]. Available: https://docs.arduino.cc/hardware/mega-2560/

[11] ——, *Arduino IDE*, Arduino SA, 2026, accessed: 2026-05-14. [Online]. Available: https://docs.arduino.cc/arduino-cli/

[12] G. F. Franklin, J. D. Powell, and M. L. Workman, *Digital Control of Dynamic Systems*, 2nd ed. Reading, MA: Addison-Wesley, 1980.

[13] RapidSCADA. (2025) What is Rapid SCADA. Accessed: Dec. 4, 2025. [Online]. Available: https://rapidscada.org/

[14] Open SCADA project. (2025) OpenSCADA. Accessed: Dec. 4, 2025. [Online]. Available: http://oscada.org/

[15] TANGO Controls. (2025) Connecting things together. Accessed: Dec. 4, 2025. [Online]. Available: https://www.tango-controls.org/